# PROJEKT

# Praktikë Profesionale

Udhëzues për zhvillimin, raportimin dhe mbrojtjen e projektit individual

| | |
|---|---|
| **Programi / Fakulteti** | Dizajn Softueri, Fakulteti i Shkencave Kompjuterike |
| **Profesori** | Prof. Ass. Dr. Endrit Fetahi |
| **Lënda** | Projekt (Praktikë Profesionale) |
| **Viti akademik** | 2026/2027 |

*Ky dokument duhet të përdoret gjatë gjithë semestrit si udhëzues për projektin, raportet teknike dhe prezantimin final.*



# Përmbajtja

1. Qëllimi dhe mënyra e zhvillimit të lëndës
2. Projekti individual dhe niveli i pritshëm
3. Organizimi i semestrit dhe raportimet
4. Raporti Teknik 1 – Analiza, kërkesat dhe dizajni teknik
5. Raporti Teknik 2 – Databaza, backend-i dhe API
6. Raporti Teknik 3 – Frontend-i, integrimi dhe progresi funksional
7. Raporti Teknik 4 – Testimi, siguria, deployment dhe versioni final
8. Projekti final dhe mbrojtja teknike
9. Git, dokumentimi dhe përdorimi i AI
10. Vlerësimi dhe rregullat e raportimit
11. Checklist për dorëzimin final

**Lexoje para se të fillosh**
Qëllimi i kësaj lënde nuk është të dorëzosh vetëm një aplikacion në fund. Projekti duhet të zhvillohet gradualisht dhe të argumentohet teknikisht në katër raportime gjatë semestrit. Secili raport mat një fazë tjetër të punës.

# 1. Qëllimi dhe mënyra e zhvillimit të lëndës

Lënda zhvillohet si punë praktike e mentoruar. Nuk parashihet model klasik i ligjëratave javore; fokusi është te zhvillimi i një projekti real, raportimi periodik i progresit dhe aftësia për ta shpjeguar punën teknike.

**Gjatë semestrit, secili student zhvillon një projekt individual.** Studenti është përgjegjës për idenë, analizën, dizajnin, implementimin, testimin, dokumentimin dhe prezantimin e projektit. Mentori jep orientim dhe feedback, por projekti duhet të jetë punë e studentit.

**Parimi kryesor**
Në vlerësim nuk merret parasysh vetëm fakti nëse aplikacioni “punon”. Vlerësohet edhe mënyra si është menduar dhe ndërtuar: arkitektura, databaza, kodi, integrimi, testimi, përdorimi i Git-it, vendimet teknike dhe aftësia për t’i mbrojtur ato.

## Çfarë pritet të arrini deri në fund?

- Të kaloni nëpër një cikël të plotë të zhvillimit të një projekti softuerik, nga ideja deri te versioni i përdorshëm.
- Të mësoni të raportoni progres teknik në mënyrë të qartë dhe të argumentuar.
- Të demonstroni jo vetëm funksionalitete, por edhe mënyrën se si funksionon sistemi “prapa skenës”.

- Të përdorni mjete profesionale si Git/GitHub dhe, sipas projektit, mjete për testim të API-ve, databazës, deployment-it ose testimit.
- Të jeni në gjendje të shpjegoni kodin dhe vendimet tuaja pa u mbështetur vetëm në slide.

# 2. Projekti individual dhe niveli i pritshëm

Të gjitha projektet janë individuale. Tema mund të zgjidhet nga studenti, me kusht që të ketë përmbajtje të mjaftueshme teknike për një projekt semestral dhe të lejojë progres të matshëm në katër fazat e raportimit.

## Çfarë lloj projekti mund të realizohet?

Projektet mund të jenë web aplikacione, mobile aplikacione, sisteme me backend dhe databazë, aplikacione desktop, platforma me API, sisteme me komponentë të inteligjencës artificiale, analiza të të dhënave të kombinuara me aplikacion funksional, ose projekte të tjera të krahasueshme për nga kompleksiteti.

**Shembull orientues**
Një student mund të zhvillojë "TaskFlow", një aplikacion për menaxhimin e projekteve dhe detyrave. Sistemi mund të ketë autentikim, role, projekte, detyra, komente, status të progresit dhe dashboard. Ky shembull përdoret disa herë në dokument vetëm për ta bërë më të qartë nivelin e pritshëm; nuk është temë që duhet kopjuar.

## Scope i arsyeshëm

Projekti duhet të jetë mjaftueshëm i madh që të demonstrojë njohuri teknike, por jo aq i madh sa të mbetet i papërfunduar. Një projekt me 5–8 funksionalitete të implementuara mirë, me databazë të strukturuar, siguri bazike, testim dhe deployment, është më i vlefshëm se një listë shumë e gjatë funksionalitetesh të papërfunduara.

| Projekt i dobët për këtë lëndë | Projekt më i përshtatshëm |
|---|---|
| Faqe statike prezantuese pa databazë dhe pa logjikë aplikative. | Aplikacion me autentikim, të dhëna persistente, logjikë biznesi dhe funksione që mund të demonstrohen. |
| Projekt i kopjuar nga një tutorial hap pas hapi. | Projekt me vendime të dokumentuara dhe implementim që studenti mund ta shpjegojë. |
| Shumë module të planifikuara, por vetëm UI pa funksionalitet. | Scope më i vogël, por funksional nga frontend-i deri në databazë. |

# 3. Organizimi i semestrit dhe raportimet

Projekti ndahet në katër raportime teknike. Raportimet bëhen gjatë semestrit, zakonisht me interval prej rreth 2–3 javësh. Datat e sakta caktohen nga profesori. Në fund zhvillohet prezantimi dhe mbrojtja finale e projektit.

| Faza | Fokusi kryesor | Çfarë duhet të ekzistojë |
|---|---|---|
| Raporti Teknik 1 | Analiza, kërkesat dhe dizajni teknik | Ide e qartë, kërkesa, arkitekturë, model fillestar i databazës, repository dhe plan pune. |
| Raporti Teknik 2 | Databaza, backend-i dhe API | Databazë reale, logjikë backend, endpoints kryesore dhe demonstrim me të dhëna reale. |
| Raporti Teknik 3 | Frontend-i dhe integrimi | Version funksional i aplikacionit, frontend-backend integration dhe demonstrim i flow-ve kryesore. |
| Raporti Teknik 4 | Testim, siguri dhe deployment | Version pothuajse final, testim, trajtim gabimesh, kontrolle sigurie dhe deployment/beta. |
| Final Project Defense | Produkti final dhe mbrojtja teknike | Live demo, kod, arkitekturë, Git history, dokumentim dhe përgjigje në pyetje teknike. |

## Formati i prezantimeve

Raportet duhet të jenë prezantime teknike, jo vetëm përmbledhje me slide. Për secilin raport rekomandohen rreth 8–12 minuta prezantim dhe disa minuta pyetje. Mund të përdoren slide për strukturë, por duhet të hapen edhe artefaktet reale të projektit: repository, kodi, databaza, API client, aplikacioni ose mjetet e testimit, varësisht nga faza.

> **E rëndësishme**
> Një screenshot nuk ka të njëjtën vlerë si një demonstrim real. Nëse thoni se një endpoint funksionon, duhet të jeni në gjendje ta thërrisni. Nëse thoni se databaza ka një relacion të caktuar, duhet ta tregoni. Nëse thoni se një funksionalitet është përfunduar, duhet ta demonstroni.

# 4. Raporti Teknik 1 – Analiza, kërkesat dhe dizajni teknik

Raporti i parë vendos bazën e projektit. Në këtë fazë nuk pritet aplikacion i përfunduar, por pritet që studenti të ketë kuptuar mirë problemin dhe të ketë marrë vendimet kryesore për mënyrën se si sistemi do të ndërtohet.

## Çfarë duhet të përmbajë raporti?

Prezantimi duhet të fillojë me problemin që zgjidh projekti dhe përdoruesit për të cilët ndërtohet. Pastaj duhet të paraqiten funksionalitetet kryesore, kërkesat funksionale dhe disa kërkesa jofunksionale. Kërkesat duhet të jenë konkrete dhe të testueshme, jo formulime të përgjithshme.

| Lloji | Shembull |
|---|---|
| Kërkesë funksionale | "Përdoruesi i autentikuar mund të krijojë një projekt, ta emërtojë dhe t'i caktojë një datë përfundimi." |
| Kërkesë jofunksionale | "Password-et ruhen vetëm në formë të hash-uar dhe endpoint-et private kërkojnë autentikim." |
| Kërkesë jofunksionale | "Interface-i duhet të jetë i përdorshëm edhe në ekran mobil për funksionet kryesore." |

Pjesa më e rëndësishme e raportit është dizajni teknik. Studenti duhet të paraqesë teknologjitë e zgjedhura dhe të shpjegojë arsyen e zgjedhjes. Duhet të ekzistojë një diagram i thjeshtë i arkitekturës së sistemit dhe një model fillestar i databazës ose strukturës së të dhënave.

```
Frontend (React / Angular / Vue / tjetër)
          ↓  HTTPS / API
Backend (Node.js / .NET / Java / Python / tjetër)
          ↓
Database (PostgreSQL / MySQL / MongoDB / tjetër)
```

Për databazën, studenti nuk duhet të tregojë vetëm emrat e tabelave. Duhet të kuptojë entitetet, primary keys, foreign keys dhe relacionet kryesore. Nëse projekti nuk përdor databazë relacionale, duhet të shpjegohet struktura alternative e të dhënave.

## Çfarë duhet të demonstrohet?

- Repository i projektit i krijuar dhe struktura fillestare e projektit.
- Diagrami i arkitekturës dhe modeli fillestar i databazës.
- Lista e funksionaliteteve që do të implementohen gjatë semestrit.
- Një plan i shkurtër se cilat pjesë do të realizohen deri në raportin e dytë.

## Shembull: TaskFlow

Në projektin shembull, studenti mund të identifikojë entitetet User, Project, Task dhe Comment. Në këtë fazë duhet të shpjegojë, për shembull, pse Task lidhet me Project, si caktohet një Task te një User dhe cilat të dhëna janë të detyrueshme. Nuk kërkohet ende që çdo funksion të jetë i implementuar, por modeli duhet të jetë i arsyetuar.

**Pyetje tipike gjatë Raportit 1**
Pse e keni zgjedhur këtë tech stack? Cili është flow kryesor i aplikacionit? Cilat janë entitetet kryesore? Pse ky relacion është one-to-many dhe jo many-to-many? Çfarë konsideroni jashtë scope-it të këtij semestri?

## Dorëzimi i Raportit 1

☐ Prezantimi i raportit.

☐ Linku i repository-t.

☐ Diagrami i arkitekturës.

☐ Diagrami/modeli fillestar i databazës.

☐ Dokument i shkurtër me kërkesat dhe funksionalitetet kryesore.

# 5. Raporti Teknik 2 – Databaza, backend-i dhe API

Në raportin e dytë projekti duhet të kalojë nga dizajni në implementim real. Fokusi është te shtresa e të dhënave dhe logjika e backend-it. Studenti duhet të jetë në gjendje të demonstrojë se si ruhen, lexohen dhe ndryshohen të dhënat dhe si komunikon aplikacioni me backend-in.

## Databaza

Duhet të paraqitet skema aktuale e databazës dhe të shpjegohen ndryshimet nga versioni fillestar. Tabelat/koleksionet kryesore duhet të ekzistojnë realisht. Studenti duhet të kuptojë constraints, lidhjet ndërmjet të dhënave dhe arsyet pse një fushë është e detyrueshme, unike ose e indeksuar, nëse aplikohet.

```
TASK
id            PRIMARY KEY
project_id    FOREIGN KEY → PROJECT.id
assigned_to   FOREIGN KEY → USER.id
title         NOT NULL
status        DEFAULT "TODO"
deadline
```

## Backend-i dhe API

Studenti duhet të ketë implementuar një pjesë domethënëse të logjikës së backend-it. Në projektet me REST API, duhet të paraqiten endpoint-et kryesore, input-i, output-i dhe status codes. Në projekte me arkitekturë tjetër, duhet të demonstrohet mekanizmi ekuivalent i komunikimit dhe logjikës.

| Method | Endpoint | Qëllimi |
|---|---|---|
| POST | /api/auth/login | Autentikimi i përdoruesit. |
| GET | /api/projects | Kthen projektet që përdoruesi ka të drejtë t'i shohë. |
| POST | /api/tasks | Krijon një detyrë të re. |
| PATCH | /api/tasks/:id | Përditëson statusin ose të dhënat e një detyre. |
| DELETE | /api/tasks/:id | Fshin një detyrë sipas rregullave të sistemit. |

Nuk mjafton të demonstrohet vetëm rasti kur gjithçka është në rregull. Të paktën një pjesë e raportit duhet të tregojë validim dhe trajtim të gabimeve. Për shembull: çfarë ndodh kur mungon një fushë, kur kërkohet një ID që nuk ekziston, ose kur një përdorues i paautorizuar provon të ndryshojë një resurs.

```
HTTP 404
{
  "success": false,
  "error": "TASK_NOT_FOUND",
  "message": "Task does not exist."
}
```

## Çfarë duhet të demonstrohet?

- Databaza reale me të dhëna testuese dhe relacionet kryesore.
- Funksionet CRUD ose logjika ekuivalente për modulet kryesore.
- Autentikimi/autorizimi bazik, nëse projekti ka përdorues dhe role.
- Thrritje reale të API-së me Postman, Insomnia, Swagger ose mjet të ngjashëm.
- Git history që tregon progres nga Raporti 1 deri në Raportin 2.

**Pyetje tipike gjatë Raportit 2**
Çfarë ndodh nga momenti kur vjen request-i deri te ruajtja në databazë? Si parandalohet një e dhënë e pavlefshme? Pse kjo lidhje në databazë është modeluar kështu? Si e kontrolloni që një përdorues nuk mund të ndryshojë të dhënat e dikujt tjetër?

## Dorëzimi i Raportit 2

☐ Prezantimi teknik me demonstrim real.

☐ Versioni aktual i repository-t.

☐ Skema e përditësuar e databazës.

☐ Lista/dokumentimi i endpoint-eve ose ndërfaqeve kryesore.

☐ Shënim i shkurtër për problemet teknike të hasura dhe mënyrën si janë zgjidhur.

# 6. Raporti Teknik 3 – Frontend-i, integrimi dhe progresi funksional

Në këtë fazë projekti duhet të fillojë të sillet si një produkt i përdorshëm. Fokusi është te integrimi i komponentëve dhe te demonstrimi i flow-ve të plota: nga veprimi i përdoruesit në UI, te backend-i dhe deri te databaza.

## Frontend dhe organizimi i kodit

Studenti duhet të paraqesë mënyrën si është organizuar frontend-i dhe të shpjegojë komponentët, faqet, shërbimet ose strukturën ekuivalente të projektit. Nuk kërkohet një arkitekturë e komplikuar vetëm për hir të kompleksitetit; kërkohet organizim i arsyeshëm dhe i kuptueshëm.

```
src/
├── components/
├── pages/
├── services/
├── hooks/
├── utils/
└── routes/
```

## Demonstrimi i një flow-i të plotë

Studenti duhet të zgjedhë të paktën një funksionalitet kryesor dhe ta shpjegojë nga fillimi deri në fund. Për shembull, te krijimi i një detyre: përdoruesi plotëson formën, bëhet validimi, dërgohet request-i, backend-i e përpunon, të dhënat ruhen dhe frontend-i përditësohet.

```
User action → Form validation → API request → Backend logic
          → Database operation → API response → UI update
```

Gjatë raportit duhet të hapet edhe kodi dhe të shpjegohet një pjesë që studenti e konsideron të rëndësishme: p.sh. authentication flow, një service, një query, menaxhimi i state-it, një algoritëm, validimi ose një integrim me shërbim të jashtëm.

## Çfarë duhet të demonstrohet?

- Të paktën disa flow-e kryesore funksionale nga UI deri në ruajtjen/leximin e të dhënave.
- Integrimi real frontend-backend ose integrimi ekuivalent për arkitekturën e zgjedhur.
- Trajtimi i loading, empty state dhe error state aty ku ka kuptim.
- Një code review i shkurtër i një pjese të rëndësishme të kodit.
- Git history me commit-e që pasqyrojnë progres të vazhdueshëm.

**Pyetje tipike gjatë Raportit 3**
Çfarë ndodh teknikisht kur klikohet ky buton? Ku bëhet validimi? Ku ruhet state-i? Çfarë ndodh nëse API dështon? Pse kjo logjikë është vendosur në këtë komponent/service dhe jo diku tjetër? Cili ka qenë bug-u më i vështirë deri tani?

## Dorëzimi i Raportit 3

☐ Prezantimi me live demo të funksionaliteteve kryesore.

☐ Versioni aktual i kodit në repository.

☐ Një diagram ose shpjegim i flow-it të një funksionaliteti kryesor.

☐ Përmbledhje e shkurtër e çfarë është përfunduar dhe çfarë mbetet deri në versionin beta.

# 7. Raporti Teknik 4 – Testimi, siguria, deployment dhe versioni final

Raporti i katërt zhvillohet kur projekti është afër përfundimit. Qëllimi nuk është të prezantohen shumë funksione të reja, por të tregohet se projekti është stabilizuar, testuar dhe përgatitur për dorëzim final.

## Testimi

Studenti duhet të demonstrojë një qasje të qartë ndaj testimit. Lloji i testimit varet nga projekti, por duhet të ketë evidencë konkrete: test cases, unit/integration tests, API tests, ose testim të strukturuar manual. Duhet të paraqiten jo vetëm testet që kalojnë, por edhe raste kufitare ose gabime që janë zbuluar dhe rregulluar.

| ID | Skenari | Rezultati i pritur | Statusi |
|---|---|---|---|
| T01 | Login me kredenciale të sakta | Përdoruesi autentikohet dhe ridrejtohet. | PASS |
| T02 | Login me password të gabuar | Kthehet gabim i kontrolluar; nuk krijohet session/token. | PASS |
| T03 | Krijim task pa titull | Request refuzohet nga validimi. | PASS |
| T04 | User A tenton të ndryshojë task të User B | Kërkesa refuzohet si e paautorizuar. | PASS |

## Siguria dhe konfigurimi

Studenti duhet të kontrollojë minimumin e nevojshëm të sigurisë për projektin: password-et nuk ruhen në plain text; secrets/API keys nuk vendosen në repository; input-i validohet; autorizimi bëhet në backend kur nevojitet; dhe të dhënat e ndjeshme nuk ekspozohen pa arsye. Nëse ndonjë prej këtyre pikave nuk aplikohet për projektin, studenti duhet të jetë në gjendje të shpjegojë pse.

## Deployment dhe versioni beta

Nëse natyra e projektit e lejon, aplikacioni duhet të jetë i deploy-uar dhe i qasshëm jashtë ambientit lokal. Për aplikacionet që nuk mund të deploy-ohen publikisht, duhet të ekzistojë një mënyrë e riprodhueshme për ta ekzekutuar projektin, me udhëzime të qarta për instalim dhe konfigurim.

> **Localhost nuk është deployment**
> Në projektet web/cloud, versioni final duhet idealisht të jetë i qasshëm përmes një URL-je. Nëse kjo nuk është e mundur për arsye teknike ose financiare, duhet të dokumentohet qartë si ekzekutohet projekti dhe cilat janë varësitë.

## Çfarë duhet të demonstrohet?

- Version beta ose pothuajse final i projektit.

- Test cases dhe/ose teste të automatizuara të përshtatshme për projektin.
- Të paktën disa gabime reale të gjetura dhe mënyra si janë rregulluar.
- Kontrollet bazike të sigurisë dhe konfigurimit.
- Deployment ose procedurë e dokumentuar për ekzekutim.
- Lista e fundit e punëve që mbeten para prezantimit final.

**Pyetje tipike gjatë Raportit 4**
Si e dini që ky funksionalitet punon edhe në rastet jo ideale? Çfarë keni testuar? Cili bug ka ndryshuar mënyrën si e keni implementuar një pjesë? Ku ruhen secrets? Si kontrollohen permissions? Çfarë duhet të bëjë dikush tjetër për ta ekzekutuar projektin tuaj?

## Dorëzimi i Raportit 4

☐ Prezantimi teknik me versionin beta.

☐ Repository i përditësuar.

☐ Evidencë të testimit.

☐ URL e deployment-it ose udhëzime të sakta për ekzekutim.

☐ Përmbledhje e problemeve të hapura para finales.

# 8. Projekti final dhe mbrojtja teknike

Prezantimi final është demonstrimi i produktit të përfunduar dhe njëkohësisht mbrojtje teknike e punës së bërë gjatë semestrit. Qëllimi është të tregoni se e zotëroni projektin nga ana funksionale dhe teknike.

## Struktura e rekomanduar e prezantimit final

| Pjesa | Përmbajtja | Koha orientuese |
|---|---|---|
| 1. Konteksti | Problemi, përdoruesi dhe zgjidhja. | 2–3 min |
| 2. Arkitektura | Komponentët kryesorë, databaza dhe vendimet teknike. | 3–4 min |
| 3. Live Demo | Flow-et kryesore dhe funksionalitetet e përfunduara. | 6–8 min |
| 4. Code / Technical Review | Një ose dy pjesë të rëndësishme të implementimit. | 3–5 min |
| 5. Pyetje teknike | Mbrojtja e projektit dhe vendimeve. | sipas nevojës |

Në finalen teknike mund t'ju kërkohet të hapni një skedar të kodit, një tabelë të databazës, një endpoint, një commit ose një konfigurim dhe ta shpjegoni. Prandaj projekti nuk duhet vetëm të jetë funksional; ju duhet ta njihni dhe ta kuptoni.

## Çfarë vlerësohet te projekti final?

- Nëse zgjidhja e përfunduar adreson problemin e deklaruar në fillim.
- Niveli i implementimit dhe funksionimi real i moduleve kryesore.
- Cilësia dhe organizimi teknik i projektit.
- Stabiliteti, trajtimi i gabimeve dhe testimi.
- Aftësia për ta shpjeguar arkitekturën, databazën dhe kodin.
- Progresi i dokumentuar gjatë semestrit dhe konsistenca me raportet teknike.

**Finalja nuk i zëvendëson raportet**
Një projekt shumë i mirë i dorëzuar vetëm në fund nuk mund të marrë pikët maksimale. Pjesë e objektivave të lëndës është puna e vazhdueshme, raportimi teknik dhe marrja e feedback-ut gjatë semestrit.

# 9. Git, dokumentimi dhe përdorimi i AI

Këto elemente vlejnë gjatë gjithë semestrit dhe kontrollohen në raportime dhe në finalen teknike.

### Git / GitHub

Repository duhet të përdoret që nga fillimi i projektit. Commit-et duhet të pasqyrojnë progres real, jo vetëm një upload të projektit në fund. Nuk kërkohet numër artificial i commit-eve, por duhet të ekzistojë histori e besueshme e punës gjatë semestrit.

| Commit jo i dobishëm | Commit më i qartë |
|---|---|
| update | feat: add project creation form |
| final | fix: validate duplicate email during registration |
| changes | refactor: move task queries to repository layer |
| test | test: add login validation cases |

### README dhe dokumentimi

Në fund repository duhet të përmbajë README që shpjegon shkurt projektin, teknologjitë, mënyrën e instalimit/ekzekutimit, konfigurimin e nevojshëm dhe, kur aplikohet, linkun e versionit të deploy-uar. Dokumentimi nuk duhet të jetë i gjatë vetëm për të shtuar faqe; duhet t'i mundësojë një personi tjetër ta kuptojë dhe ta ekzekutojë projektin.

## 10. Vlerësimi dhe rregullat e raportimit

Vlerësimi ndahet ndërmjet progresit gjatë semestrit dhe cilësisë së projektit final. Katër raportet kanë peshë të barabartë, në mënyrë që secila fazë e projektit të ketë rëndësi.

| Komponenti | Pikët |
|---|---|
| Raporti Teknik 1 – Analiza, kërkesat dhe dizajni teknik | 10 |
| Raporti Teknik 2 – Databaza, backend-i dhe API | 10 |
| Raporti Teknik 3 – Frontend-i, integrimi dhe progresi funksional | 10 |
| Raporti Teknik 4 – Testimi, siguria, deployment dhe beta | 10 |
| Projekti final dhe live demo | 40 |
| Mbrojtja teknike, Git dhe dokumentimi | 20 |
| TOTAL | 100 |

## Maksimumi i pikëve sipas raportimeve të realizuara

Raportet duhet të realizohen në kohën e caktuar. Nëse një student nuk i realizon raportet gjatë semestrit, projekti final ende mund të vlerësohet, por nuk mund të rikuperojë pikët e humbura nga procesi i raportimit.

| Raportet e realizuara | Pikë nga raportet | Maksimumi i mundshëm në lëndë |
|---|---|---|
| 4/4 | 40/40 | 100/100 |
| 3/4 | 30/40 | 90/100 |
| 2/4 | 20/40 | 80/100 |
| 1/4 | 10/40 | 70/100 |
| 0/4 | 0/40 | 60/100 |

Shembull: një student që ka realizuar 2 nga 4 raportet mund të ketë maksimumi 80 pikë, edhe nëse projekti final është shumë i mirë. Kjo sepse 20 pikët e dy raporteve të parealizuara nuk transferohen te finalja.

## Si vlerësohet secili raport teknik?

| Kriteri | Çfarë kërkohet |
|---|---|
| Përgatitja teknike | Materiali i kërkuar për fazën ekziston dhe është i përditësuar. |
| Demonstrimi | Studenti tregon artefaktet reale dhe jo vetëm slide/screenshot. |
| Kuptimi | Studenti mund t'i shpjegojë vendimet, kodin, databazën dhe flow-in relevant. |
| Progresi | Ka progres të dukshëm nga raporti paraprak dhe Git history e mbështet atë. |
| Pyetjet | Studenti përgjigjet në mënyrë të arsyetuar në pyetjet teknike të fazës. |

## Raportet e vonuara ose të parealizuara

Një raport i parealizuar në afatin e caktuar konsiderohet i humbur për pikët e asaj faze, përveç nëse profesori lejon ndryshe për një arsye të justifikuar. Edhe kur një raport i vonuar pranohet për feedback, kjo nuk nënkupton automatikisht rikthimin e pikëve. Qëllimi është që puna të zhvillohet dhe të prezantohet gjatë semestrit, jo vetëm në fund.

## Origjinaliteti dhe integriteti akademik

Projektet duhet të jenë punë individuale. Mund të përdoren biblioteka, dokumentacion, tutoriale dhe mjete ndihmëse, por nuk lejohet paraqitja e një projekti të marrë nga një student tjetër, repository tjetër

ose burim i jashtëm si punë personale. Në rast dyshimi, studenti mund të pyetet në detaje ose t'i kërkohet të ndryshojë/implementojë një pjesë gjatë mbrojtjes.

# 11. Checklist për dorëzimin final

Para prezantimit final, kontrolloni që projekti juaj të jetë në një gjendje që mund të demonstrohet dhe kuptohet nga një person tjetër.

- ☐ Projektin mund ta ekzekutoj nga një ambient i pastër duke ndjekur README.
- ☐ Repository përmban versionin final dhe histori të rregullt të commit-eve.
- ☐ Nuk kam password-e, API keys ose secrets të ekspozuara në repository.
- ☐ Databaza/skema është e përditësuar dhe mund ta shpjegoj.
- ☐ Funksionalitetet kryesore punojnë në live demo.
- ☐ Kam testuar rastet kryesore dhe disa raste gabimi.
- ☐ Mund ta shpjegoj një flow nga UI deri në databazë.
- ☐ Mund të hap kodin dhe ta shpjegoj pa u mbështetur vetëm në slide.
- ☐ Kam përgatitur linkun e deployment-it ose udhëzimet e ekzekutimit.
- ☐ Kam dokumentuar qartë kufizimet ose funksionalitetet që nuk janë përfunduar.

**Në ditën e finales**
Mos e ndërtoni prezantimin rreth slide-ve. Prezantimi duhet të mbështetë demonstrimin e projektit. Hapni paraprakisht aplikacionin, repository-n, databazën dhe mjetet që mund t'ju duhen, në mënyrë që koha të shfrytëzohet për demonstrim dhe diskutim teknik.

**Qëllimi është që në fund të semestrit të mos keni vetëm një projekt për notë, por një projekt që mund ta prezantoni si dëshmi konkrete të punës dhe aftësive tuaja teknike.**